RAG explained from zero

Retrieval-Augmented Generation, the corpus, chunking, embeddings, golden sets and graphs, for someone new to AI.

With how the Knowledge Portal applies each one, and why.

Knowledge Portal | 15 October 2026

A learning guide. Project figures come from the repository and its tests.

Contents

1. Start here: what an LLM is **5**

The three limits that RAG fixes . . . 5

2. What RAG is **6**

The two phases . . . 6

One question, start to finish . . . 7

3. When to recommend RAG, and when not **8**

Recommend RAG when . . . 8

Prefer something else when . . . 8

4. The corpus: what RAG searches **9**

What makes a good corpus . . . 9

Corpus, index, golden set: three different things . . . 9

5. Inside indexing: from a document to searchable chunks **10**

Step 1: extract the text . . . 10

Step 2: review (a human checks) . . . 10

Step 3: chunking, and why we do it . . . 10

Step 4: embedding . . . 12

Step 5: store in the index . . . 12

6. Inside a query: finding the right chunks **13**

Embeddings: a map of meaning . . . 13

Why we also search by keywords . . . 13

Combining two rankings: Reciprocal Rank Fusion . . . 14

Filter, diversify and rerank . . . 14

Building the prompt . . . 15

7. How RAG and the LLM fit together, and why it works **16**

What the LLM actually does . . . 16

Why this design is used so widely . . . 16

Two models, two jobs . . . 17

8. The golden set: measuring retrieval **18**

Where it sits: not inside the LLM, not inside RAG . . . 18

What it measures . . . 18

Why it matters . . . 19

9. Graphs: when similarity is not enough **20**

What a graph is . . . 20

Vector search and graphs answer different questions . . . 20
Why we added a graph beside vectors . . . 21

10. How our project applies all of this, and why **22**

Use 1: the reference library is classic RAG . . . 22
Use 2: requirement assessment combines a graph with RAG . . . 23
Use 3: golden sets gate every retrieval change . . . 24
Why we chose each piece . . . 24
What comes next . . . 25

11. Glossary **26**

How to read this guide

This guide is written for someone new to AI. It assumes you can read a little code and know what a database is, and nothing else. Each idea is introduced once, in plain words, with a picture or an everyday comparison, before it is used. Words in **bold** are collected in the glossary at the end.

The chapters build on each other:

1. **What an LLM is** and the three things it cannot do on its own.
2. **What RAG is**: the idea, the two phases, and a full example from question to answer.
3. **When to recommend RAG**, and when something else is the better tool.
4. **The corpus**: the collection of documents RAG searches.
5. **Inside indexing**: how a document becomes searchable, and why we cut it into chunks.
6. **Inside a query**: embeddings, vector search, keyword search, ranking and the prompt.
7. **How RAG and the LLM fit together**, and why this design works.
8. **The golden set**: how we measure whether retrieval is any good.
9. **Graphs**: what a graph is, and why we use one beside vectors.
10. **How our project applies all of this**, and why each choice was made.

> **Key idea**
> RAG is not a model and not a product. It is a **pattern**: search your own documents first, then give the best pieces to a language model and ask it to answer from them. Almost everything in this guide is about doing the search part well.

A note on spelling: you may have heard the word as "corbus". It is written **corpus** (plural **corpora**), from the Latin for "body": a body of text.

1. Start here: what an LLM is

A **large language model** (LLM) is a program that has read an enormous amount of text and learned to continue text. You give it some words, the **prompt**, and it predicts the most likely next word, then the next, until it has written an answer. ChatGPT, Gemini and Claude are LLMs.

It does this with billions of numbers called **weights**, tuned during **training**. Everything the model "knows" is spread across those weights. Nobody can open them and read a fact out, and once training ends they stay fixed.

> **Think of it like this**
> An LLM is like a brilliant new hire who has read every public book and website up to a certain date, but has never seen a single file from your company. They write beautifully and reason well. Ask them your bundle limit per site and they will give a confident answer, which may simply be made up.

The three limits that RAG fixes

Limit	What it means	Example
It does not know your private data	Your policies, product guides and procedures were never in its training text.	"How many bundles may an SMB customer hold per site?" It cannot know: the rule lives in one of our documents.
Its knowledge is frozen	Training stopped on a date (the **knowledge cutoff**). Anything that changed later is invisible.	A policy approved last week does not exist for the model.
It can invent answers	When unsure, it still produces fluent text. This is called **hallucination**. It also cannot tell you where an answer came from.	It cites a clause number that does not exist.

There is a fourth, practical limit. A model can only read a certain amount of text at once, its **context window**, measured in **tokens** (pieces of words, roughly 3 to 4 characters of English each). You cannot paste a whole library into every question; it would not fit, would cost a lot, and would be slow.

> **Key idea**
> Retraining the model on your documents is slow, expensive and has to be repeated whenever a document changes. RAG avoids retraining entirely. It leaves the model as it is and changes what you *show* it.

2. What RAG is

RAG stands for **Retrieval-Augmented Generation**. Read the three words backwards and you have the whole method:

- **Generation**: an LLM writes the answer.
- **Augmented**: but its prompt is first enriched with extra material...
- **Retrieval**: ...which was found by searching your own documents for the question.

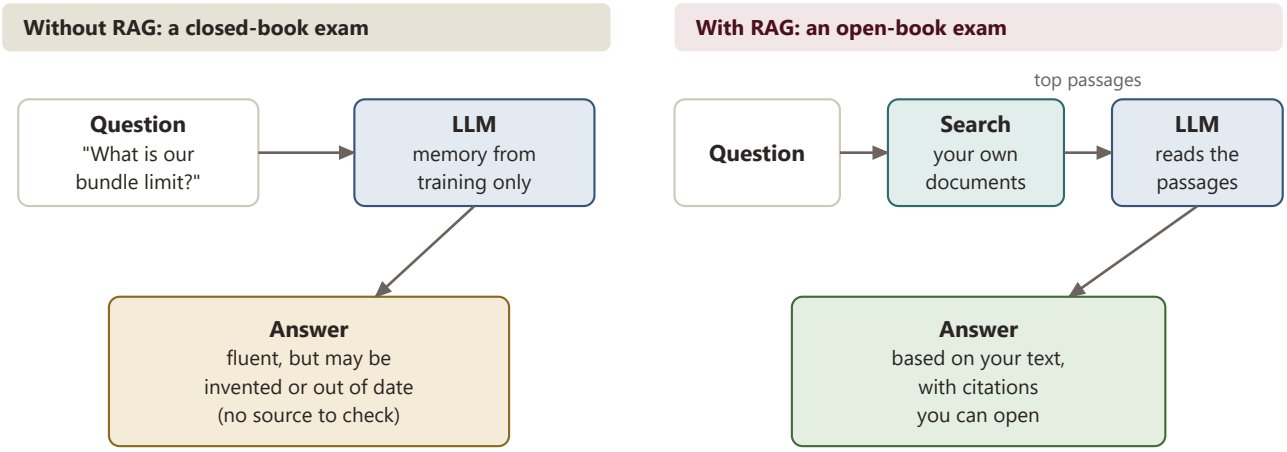


Figure 1. The same question without and with RAG. With RAG the model answers from your text and the answer carries citations someone can check.

> **Think of it like this**
> Without RAG the model sits a closed-book exam: it must answer from memory. With RAG it sits an open-book exam, but someone else (the retriever) quickly finds the right three pages and puts them on the desk. The quality of the answer now depends mostly on whether the right pages were found.

The two phases

A RAG system works in two separate phases. Keeping them apart in your head makes everything else easier.

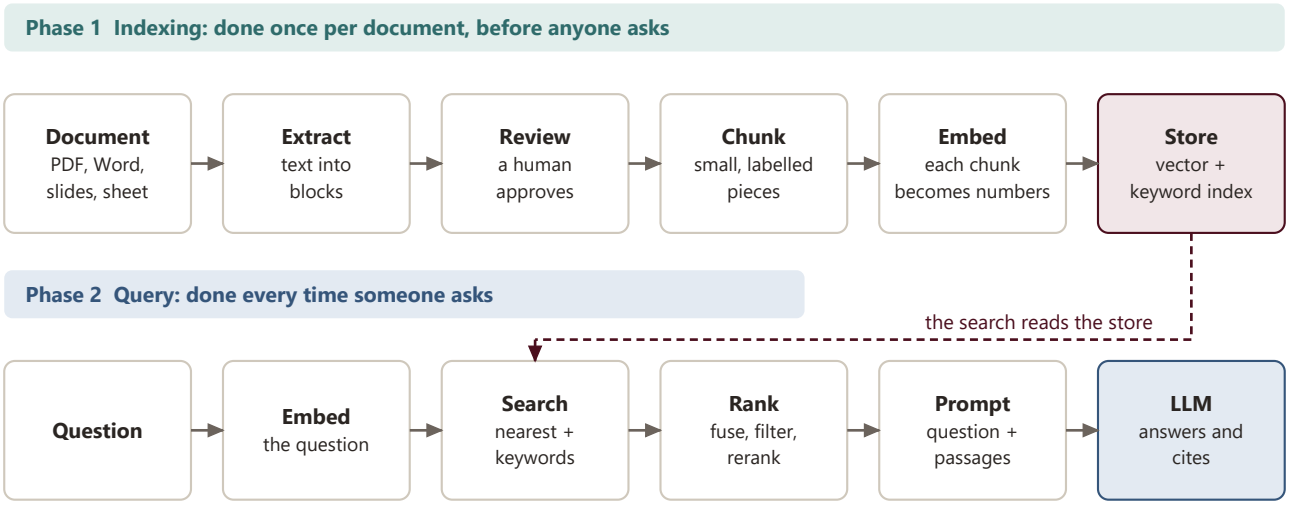


Figure 2. Indexing prepares documents once; querying runs on every question. The LLM is only the last step of the query phase; everything before it is ordinary software you control.

- **Indexing** happens when a document arrives. The text is extracted, checked, cut into small pieces called **chunks**, turned into numbers (**embeddings**) and stored in an **index**. This is slow-ish, but it happens once per document version.
- **Querying** happens every time someone asks. The question is turned into numbers the same way, the index returns the nearest chunks, the best few are placed into a prompt, and the LLM writes the answer. This must be fast.

One question, start to finish

Here is a concrete walk through, using documents that really exist in our test corpus.

1. An analyst asks: "*Must sales check fibre coverage before giving a quote?*"
2. The question is sent to the **embedding model**, which returns 768 numbers describing its meaning.
3. The index compares those numbers with every stored chunk and also runs a keyword search for words like "coverage" and "quote".
4. Two chunks come back on top: "*Confirm XGPON coverage at the customer's address before quoting.*" from the *Bundle ordering procedure*, and "*High-speed bundles require XGPON coverage at the customer address.*" from the *XGPON coverage rules*.
5. A prompt is built: the rules ("answer only from these passages, cite them"), the two passages with ids, and the question.
6. The LLM answers: "*Yes. Coverage must be confirmed at the customer's address before a quote [P1], because high-speed bundles require it [P2].*"
7. Before the answer is used, the system checks that P1 and P2 are still the approved, current versions of those documents.

Key idea

Notice that the LLM did nothing clever about *finding* the answer. The search did that. If the search had returned the wrong chunks, the LLM would have written a fluent answer from the wrong material. That is why most RAG engineering, and most of this guide, is about retrieval.

3. When to recommend RAG, and when not

RAG is the right tool when the answer is written down somewhere in a body of text that the model has not seen, and when people need to trust and check that answer.

Recommend RAG when

Situation	Why RAG fits
Knowledge is private: policies, product guides, contracts, procedures	The model never saw it. RAG shows it at question time.
Knowledge changes often	Approve a new version and it is searchable minutes later. No retraining.
Answers must be traceable	Every answer can point to the exact passage it used, so a reviewer can open it.
There is far more text than fits in a prompt	Retrieval picks the few pieces that matter.
Different people may see different documents	The search can filter by permission before anything reaches the model.
Wrong answers are costly	Grounding in real text and checking citations reduces invention. It does not remove it.

Prefer something else when

Situation	Better tool	Why
You want a different tone, format or behaviour	Better prompts, or **fine-tuning**	RAG adds knowledge; it does not change how the model writes.
All the knowledge fits in one prompt and rarely changes	Put it in the prompt directly	A search step adds moving parts for no gain.
The question needs counting or totals over many records ("how many orders failed last month?")	A database query (SQL), maybe written by an LLM	Retrieval returns a handful of passages, not every row. It cannot add up a table.
The answer is an exact lookup by id ("price of plan X")	A normal API or database	Exact data deserves an exact query, not a similarity search.
The answer depends on how things are connected ("which systems change if this data changes?")	A **graph**, often together with RAG	Similar wording does not reveal relationships. Chapter 9 explains.

> **Watch out**
>
> RAG is not a guarantee of truth. If the corpus holds an outdated or wrong document, RAG will faithfully repeat it. Curation of the documents matters as much as the technology.

4. The corpus: what RAG searches

The **corpus** is the complete set of documents a RAG system is allowed to search. It is the "book" in the open-book exam. The model never sees the whole corpus; it only ever sees the few chunks retrieved from it for one question.

> **Think of it like this**
> If RAG is a librarian who fetches the right pages, the corpus is the library. A skilled librarian cannot help you if the library holds the wrong books, old editions, or books nobody checked.

What makes a good corpus

- **Relevant**: it contains the documents people actually need answers from.
- **Correct and current**: each document is the approved version. Old versions are replaced or withdrawn, not left lying around to be retrieved.
- **Readable**: text was extracted cleanly. A scanned page with no text, or a table that came out scrambled, cannot be found by any search.
- **Owned**: someone is responsible for each document and reviews it on a schedule.
- **Permissioned**: it is clear who may read what.

Corpus, index, golden set: three different things

Term	What it is	Analogy
Corpus	The documents themselves, as approved.	The books in the library.
Index	Chunks of those documents plus their numbers, stored for fast search.	The library's catalogue cards.
Golden set	A list of test questions with the answers an expert expects. Used to measure the search, never shown to users.	The answer key used to test a new librarian.

> **In our project**
> We have two main corpora. The **reference library** holds documents that knowledge admins upload and review: policies, procedures, guides, in any format. The **architecture catalogue** holds the systems, offerings, journeys and capabilities of the business, published as numbered releases, plus the design documents behind them. Only what is *approved and in service* belongs to the corpus. Drafts and withdrawn versions are never searched.

5. Inside indexing: from a document to searchable chunks

Step 1: extract the text

Documents arrive as PDF, Word, PowerPoint, Excel, CSV, Markdown or images. A program called an **extractor** opens each format and pulls out its text as **blocks**: a paragraph, a heading, a table row, a slide paragraph, a spreadsheet row, a page. Each block remembers where it came from, its **label**, for example "Page 2", "Slide 3, paragraph 1" or "Table 1, row 3".

Extraction is where many RAG systems quietly fail. A table read as one long line, a chart that has no text, or a scanned page with no text layer cannot be found later, however good the search is.

Step 2: review (a human checks)

Many RAG systems skip this. We do not. A person compares the extracted blocks with the original and decides, block by block, what may be used. Only after approval does the document enter the corpus.

Step 3: chunking, and why we do it

Chunking means cutting the approved text into small pieces before indexing. A beginner often asks: why not store each document whole? There are four reasons.

Reason	Explanation
1. One number-list per document blurs its meaning	An embedding squeezes a text into one point on the map of meaning (Chapter 6). A 40-page guide about billing, roaming and security becomes one blurry point that is "a bit about everything" and close to nothing. A one-paragraph chunk about roaming is a sharp point that a roaming question lands right next to.
2. Embedding models have input limits	They accept a limited amount of text per call. Long documents simply do not fit.
3. The prompt has a budget	We can only show the LLM a few thousand words. Better to show the five paragraphs that matter than one random page of a long document.
4. Precise citations	A citation that says "Fleet guide, Table 1, row 3" can be checked in seconds. "Fleet guide" cannot.

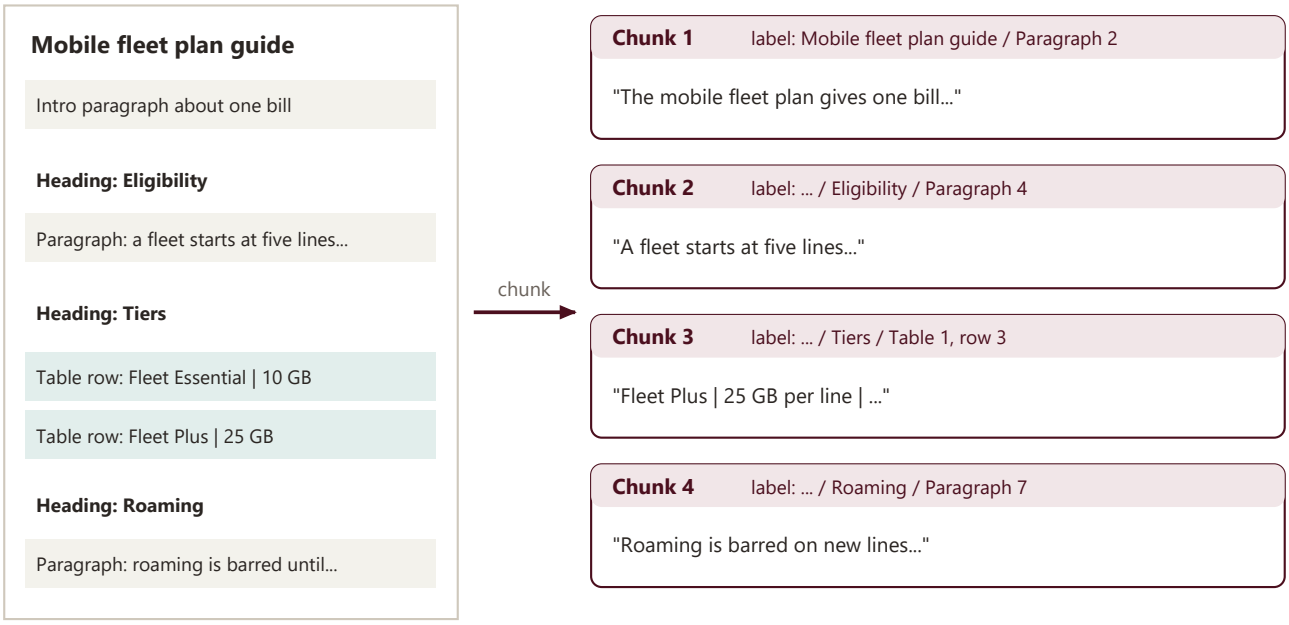


Figure 3. A guide is cut along its own structure. Each chunk carries a label with the document title, section and location, so it can be cited exactly.

How big should a chunk be?

It is a trade-off. Too small, and a chunk loses the context that gives it meaning ("it is barred until enabled": what is?). Too big, and the meaning blurs again and the prompt fills up with text that does not help. Common sizes are a paragraph or a few hundred words.

Strategy	How it cuts	Good for
Fixed size	Every N characters, regardless of content.	Quick experiments. It often cuts sentences in half.
Sentence-aware	Fills up to a size limit, but only breaks at sentence ends.	Plain prose.
Structure-aware	Follows the document's own structure: headings, paragraphs, table rows, slides; then sentence-aware inside a long block.	Real business documents. This is what we use.
With context window	Stores small chunks for search but also keeps their neighbours, and shows the neighbours to the LLM.	Getting precise search *and* enough context. We do this too.

In our project

Our chunker never mixes two blocks in one chunk, prefers to break at sentence ends, and aims for about 512 and at most 768 bytes of text per chunk (bytes, not tokens, so Arabic and English are budgeted safely). Each chunk's label starts with the document title and its section. For the prompt, a chunk brings its neighbours from the same section, up to about 1,536 bytes, so the model sees the sentence before and after.

Step 4: embedding

Each chunk is sent to an **embedding model**, which returns a list of numbers, a **vector**. Ours has 768 numbers. Chapter 6 explains what those numbers mean. For now: two texts with similar meaning get similar lists.

Step 5: store in the index

Each chunk is saved with its text, its label, where it came from, and its vector. Two search structures are built on top:

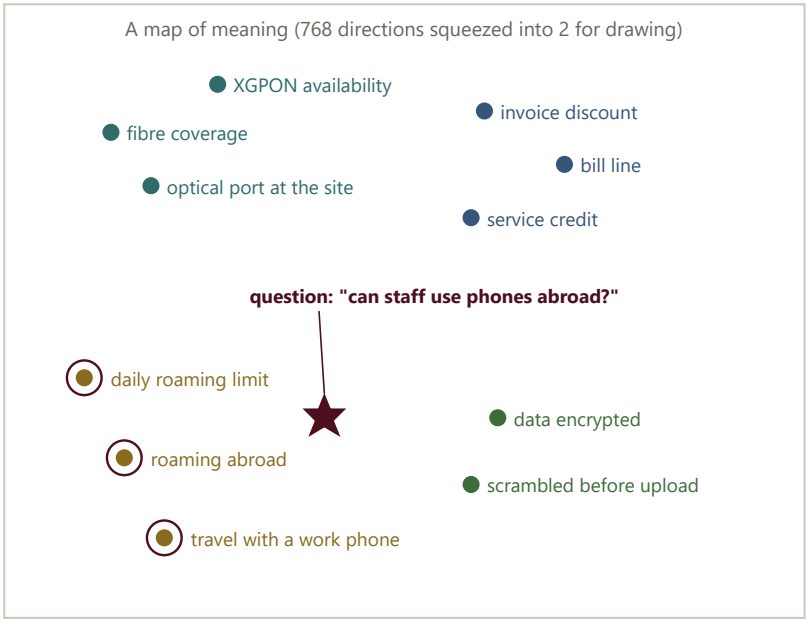
- A **vector index** for "find the nearest vectors". Comparing a question with millions of vectors one by one is slow, so a shortcut structure called **HNSW** (a layered map of neighbours) finds the nearest ones quickly, with a tiny chance of missing one.
- A **keyword index** for "find chunks containing these words", like a book's index at the back.

> **In our project**
> Both live in our PostgreSQL database. The **pgvector** extension stores the 768-number vectors with an HNSW index, and PostgreSQL's own text search (tsvector) is the keyword index. One database holds documents, chunks and vectors, so there is no separate vector product to run and keep in step.

6. Inside a query: finding the right chunks

Embeddings: a map of meaning

Imagine a huge map where every piece of text is a dot. The embedding model decides where each dot goes, and it has learned, from billions of sentences, to put texts with similar meaning near each other. Real maps have 2 directions; ours has 768, which lets it keep apart many different kinds of similarity at once.



How to read it

Every chunk and every question is turned into a long list of numbers: its embedding. Texts that mean similar things land close together, even with different words.

Searching means: put the question on the map and take the chunks nearest to it (ringed). Here the roaming chunks win, although "abroad" and "travel" are different words.

Closeness is usually cosine similarity: the angle between two arrows from the centre. 1 = same direction, 0 = unrelated.

Figure 4. Texts about the same topic cluster together. A question lands on the same map, and its nearest chunks are the search result.

To measure "near", we usually use **cosine similarity**: think of each vector as an arrow from the centre of the map, and measure the angle between two arrows. Same direction gives 1, unrelated gives about 0. The search returns the chunks with the highest similarity: the **top-k**, where k is how many we want.

> **Key idea**
>
> The question must be embedded by **the same model** as the chunks. Two different embedding models draw two different maps; comparing dots across them is meaningless. That is why changing the embedding model means re-indexing the whole corpus.

Why we also search by keywords

Vector search is great with meaning and weak with exact strings. Keyword search is the opposite. Using both is called **hybrid search**.

Question	Vector search	Keyword search
"Is data scrambled before it leaves the site?" (the document says "encrypted")	Finds it: same meaning.	Misses it: no shared word.
"Rules for SIP Trunk 30"	May confuse it with SIP Trunk 10: the meanings are almost the same.	Finds the exact name.

Question	Vector search	Keyword search
"XGPON" (a product code)	Unreliable: a rare code has little learned meaning.	Finds every chunk that contains it.

Combining two rankings: Reciprocal Rank Fusion

Each search returns its own ordered list. **Reciprocal Rank Fusion** (RRF) merges them with a simple rule: a chunk earns 1 / (60 + its rank) from each list it appears in, and the totals decide the final order. The 60 softens the gap between first and second place, so a chunk that both searches like beats one that only one search loves.

Chunk	Keyword rank	Vector rank	RRF score	Final
A	1	2	1/61 + 1/62 = 0.0325	1st
C	3	1	1/63 + 1/61 = 0.0323	2nd
B	2	not found	1/62 = 0.0161	3rd
D	not found	3	1/63 = 0.0159	4th

RRF needs no tuning and does not care that the two searches score on different scales, which is why it is a popular default.

Filter, diversify and rerank

- **Filter**: drop anything the asker may not see, and anything that is no longer the approved, current version.
- **Diversify**: stop one long document from filling every slot. We keep at most 3 chunks per document and at most 10 documents per answer.
- **Rerank** (optional): take the top few dozen and reorder them with a slower, more careful scorer. A **cross-encoder** reads the question and each passage together, which is more accurate than comparing two separate vectors.

Building the prompt

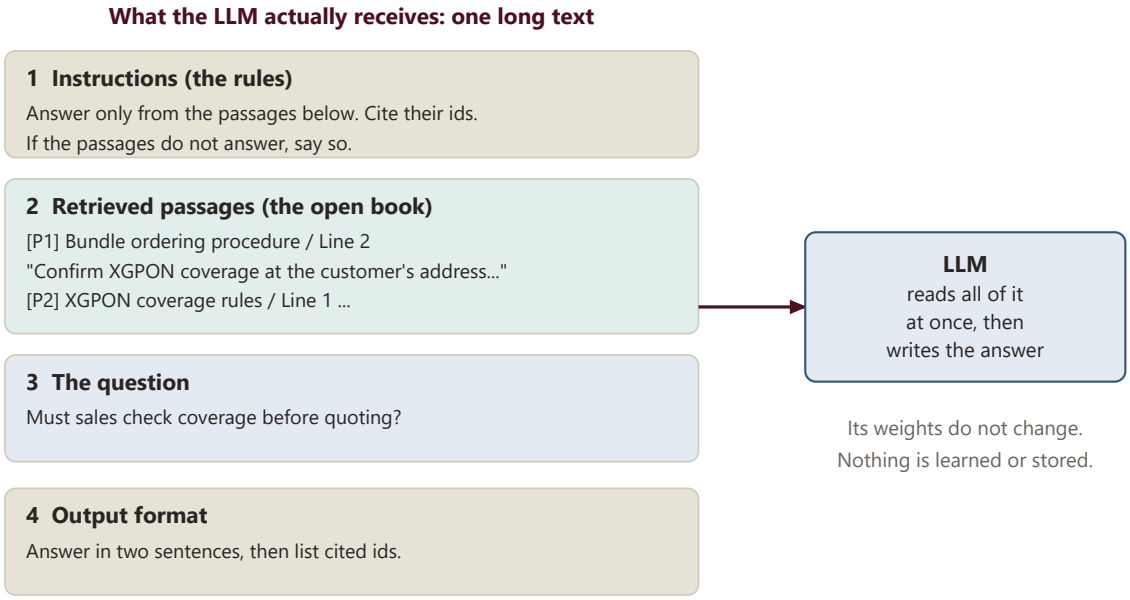
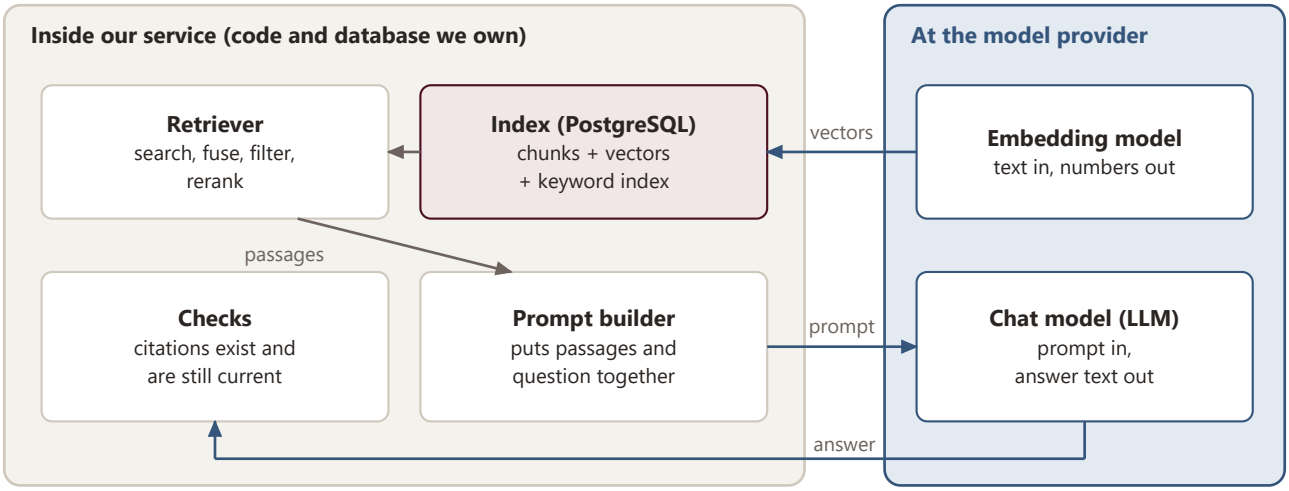


Figure 5. The prompt is just text: rules, the retrieved passages with ids, the question and the expected format.

The passages are placed inside the prompt with short ids so the model can cite them. The instructions tell it to answer only from them and to say so when they do not answer. After the model replies, good systems **check the citations**: does the quoted text really appear in the cited passage, and is that passage still current?

7. How RAG and the LLM fit together, and why it works



Two different models: one that measures meaning, one that writes. RAG glues them together with ordinary code.

Figure 6. The retriever, the index, the prompt builder and the checks are our own code. The models are called over the network.

What the LLM actually does

The LLM receives one long text (Figure 5) and writes a continuation. Inside the model, a mechanism called **attention** lets every word of the answer "look back" at every word of the prompt and weigh which ones matter. When the right passage is in the prompt, attention lets the model copy facts from it and phrase them well. This is a skill the model already has from training: reading a text and answering questions about it.

> **Key idea**
> Nothing is learned. The model's weights do not change, and after the answer is written it remembers nothing of your documents. Every question starts fresh, with whatever the retriever put in front of it.

Why this design is used so widely

Benefit	Because
Fresh	Updating knowledge means updating the index, which takes minutes, not retraining a model, which takes weeks and a lot of money.
Traceable	Every answer can list the passages it used. People can verify instead of trust.
Controllable	You decide what is in the corpus, who can see it, and when a document is withdrawn.
Model-independent	You can switch to a better LLM tomorrow. The corpus and index stay; only the embedding model change forces a re-index.
Cheaper	Only a few passages are sent per question, instead of whole documents.

Two models, two jobs

A RAG system uses at least two different models. The **embedding model** turns text into vectors and is used both when indexing and when querying. The **chat model** (the LLM) only writes the answer. They are trained differently, priced differently, and can come from different providers.

> **In our project**
>
> Both are reached through **ports**, small interfaces in our code, so the rest of the system never knows which provider is behind them. A configuration file (config/llm.yaml) picks the provider; today the embedding is Google's gemini-embedding-001 with 768 dimensions. For tests there is a **fake** provider that runs offline: its "embedding" hashes words into 768 numbers, so it matches shared words but understands no meaning. That lets every test run without internet or cost.

8. The golden set: measuring retrieval

A **golden set** is a list of test questions, each paired with the answer an expert says is right. For retrieval, "the answer" means: which passages should come back. It is also called an evaluation set, a test set or ground truth.

> **Think of it like this**
> When a new librarian starts, you do not judge them by how confident they sound. You hand them twenty questions whose right books you already know, and you count how many they fetch. The golden set is that list of twenty questions and the answer key.

Where it sits: not inside the LLM, not inside RAG

The golden set is not part of the model, and it is not used when a real user asks a question. It sits *outside* the pipeline, like an inspector. Developers run it before and after every change, and the test suite runs it automatically so a change that makes retrieval worse is caught before it ships.

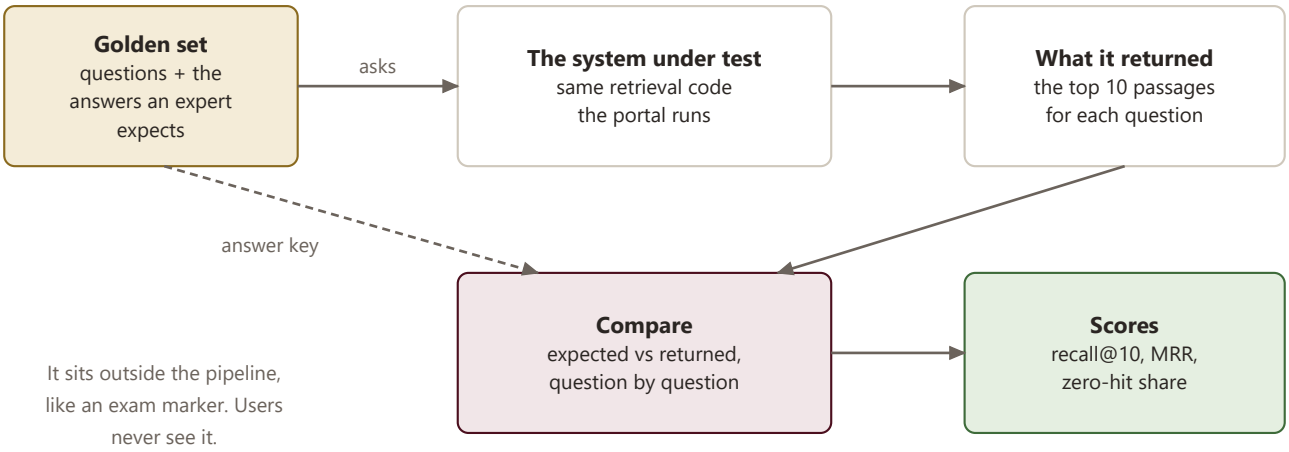


Figure 7. The golden set feeds known questions to the real retrieval code and compares what comes back with the answer key.

What it measures

Three numbers answer three different questions:

Score	The question it answers	How it is computed
Recall@10	Did the right passages appear at all in the top 10?	For each question: expected passages found in the top 10, divided by expected passages. Then the average over questions.
Mean reciprocal rank (MRR)	How high did the first right passage appear?	For each question: 1 divided by the rank of the first right passage (1st = 1, 2nd = 0.5, 4th = 0.25, missing = 0). Then the average.
Zero-hit share	How often did we find nothing useful?	The share of questions with no right passage in the top 10.

A worked example with three questions

Question	First right passage at	Recall@10	Reciprocal rank	Zero hit?
Q1	rank 1	1 of 1 = 1.0	1/1 = 1.00	no
Q2	rank 4	1 of 1 = 1.0	1/4 = 0.25	no
Q3	not in top 10	0 of 1 = 0.0	0	yes
Result		**(1+1+0)/3 = 67%**	**(1+0.25+0)/3 = 0.42**	**1 of 3 = 33%**

Recall tells you whether the LLM *could* have answered. MRR tells you whether the answer was near the top, where it is most likely to be used. Zero hits are the cases where the LLM had nothing to work with and was most likely to invent.

Why it matters

- **You cannot improve what you do not measure.** Chunk size, the embedding model, keyword settings and rerankers all have knobs. Without a golden set, every change is a guess.
- **It catches regressions.** A change that helps one question may break five others. The golden set shows both.
- **It turns opinions into numbers.** "Search feels better" becomes "recall rose from 90% to 95%".
- **It finds real problems.** Ours showed that a document's own prose can push its table out of the results (Chapter 10).

> **Watch out**
>
> A golden set is only as good as its questions. Keep it honest: write questions the way real users ask them, include hard ones that share no words with the answer, never tune the system on the exact questions you report on, and label invented questions as fixtures rather than evidence from real users.

9. Graphs: when similarity is not enough

What a graph is

In computing, a **graph** is not a chart. It is a set of **nodes** (things) joined by **edges** (named relationships). A **knowledge graph** is a graph of business facts: this offering *has* that component, which *is fulfilled by* that system, which *sends orders to* another system.

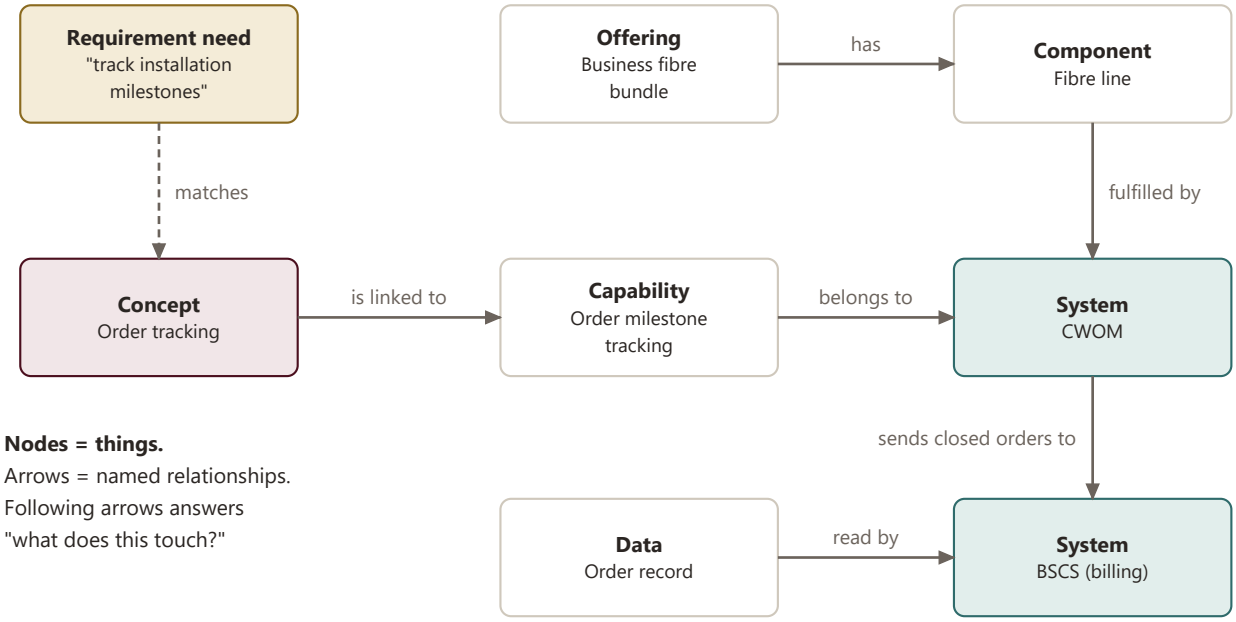


Figure 8. A small part of a catalogue graph. Following the arrows from a business need leads to the systems it touches, and each arrow explains a step.

> **Think of it like this**
> Vector search is like asking "which books sound like my question?". A graph is like a family tree or a metro map: it answers "who is connected to whom, and how?". You would never find your second cousin by looking for people whose names sound like yours.

Vector search and graphs answer different questions

	Vector search	Graph
Stores	Text chunks as points on a map of meaning.	Things and the relationships between them.
Best question	"What text is about this?"	"What is connected to this, and how?"
Strength	Understands paraphrases; needs no manual modelling.	Exact, multi-step and explainable: every step is a named link.
Weakness	Cannot follow relationships; similar wording is not a dependency.	Someone must build and maintain it; it only knows what was entered.
Typical use	Finding passages to quote.	Impact analysis, ownership, "what breaks if this changes".

Why we added a graph beside vectors

We did **not** replace vectors with a graph; we use both. The graph was added for one job where similarity alone failed: deciding *which systems a requirement will change*.

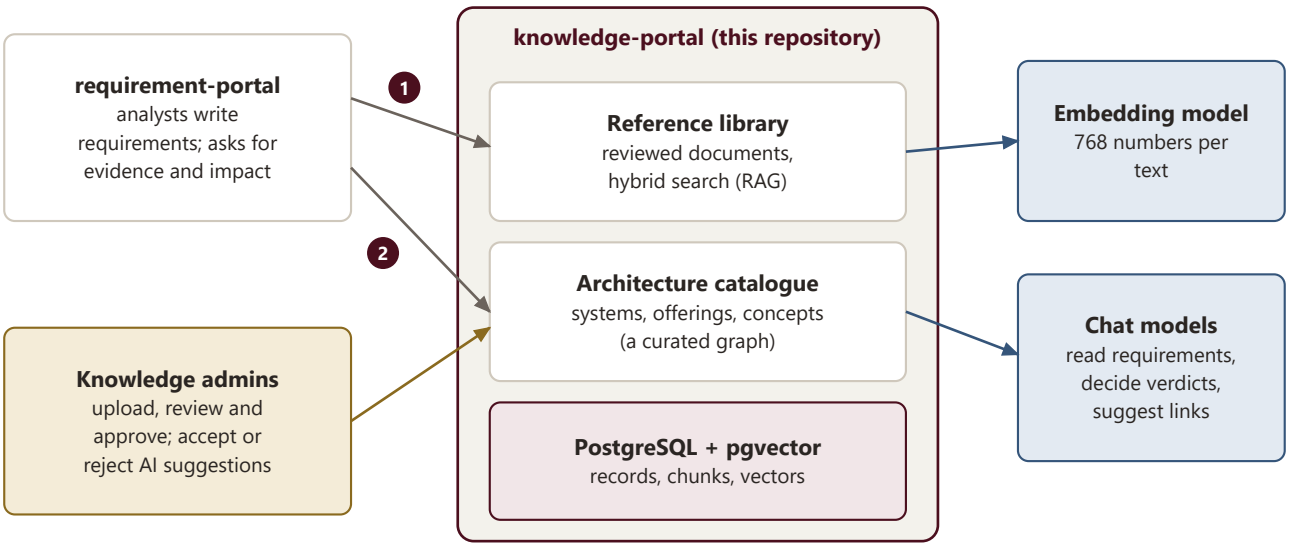
1. **Requirements describe needs, not systems.** "Let customers see installation milestones" never says "CWOM". Searching for similar text rarely lands on the right system record.
2. **We measured it.** On our golden set, matching by names and embeddings found only about 8% of the systems a reviewer expected (ADR-0114).
3. **Impact follows relationships.** If a requirement changes an order record, every system that *reads* that record must be checked. That is a walk along edges, and no amount of text similarity reveals it.
4. **Reviewers need a reason.** A graph gives a path: need, then concept "Order tracking", then capability, then system CWOM. Each step can be checked.

> **In our project**
>
> Our graph is **curated**: every link between a system and a business capability concept is accepted by a maintainer before it counts. We considered letting an LLM extract a graph from documents and rejected it (ADR-0114), because links nobody reviewed would then decide which systems change.

You will also meet the term **GraphRAG**: any RAG system that uses a graph during retrieval. Ours is one such design: the graph finds the systems, and ordinary RAG finds the passages that justify them.

10. How our project applies all of this, and why

Our platform has three repositories. **requirement-portal** is where analysts write and analyse requirements. **knowledge-portal**, this one, holds and curates the knowledge they rely on. **platform-kernel** holds shared building blocks. The requirement portal never reads our database; it calls our internal API with a service token.



1 /internal/library/retrieve: passages for a question. 2 /internal/architecture/assess: systems and verdict for a requirement.

Figure 9. The knowledge portal serves two kinds of retrieval to the requirement portal, both from one PostgreSQL database.

Use 1: the reference library is classic RAG

Knowledge admins upload documents in any format. Each passes through the full indexing phase of Chapter 5:

Stage	What happens in our library
Upload	Any supported format: PDF, Word, PowerPoint, Excel, CSV, text, Markdown, images. Each upload is scanned for malware first.
Extract	The kernel's extractors turn it into labelled blocks (pages, paragraphs, table rows, slides, worksheet ranges).
Review	The owner includes or excludes every block, including images, and approves the revision. Blocks with blocking extraction problems cannot be published.
Chunk	Structure-aware chunks of about 512 to 768 bytes, each labelled with title and section, with a neighbour window for context.
Embed and store	A background worker embeds the chunks (768 numbers each) and stores them in library_chunks: pgvector with an HNSW index, plus a keyword index.
Search	Hybrid search with Reciprocal Rank Fusion, only the version in service, at most 3 chunks per document and 10 documents.
Serve	requirement-portal calls /internal/library/retrieve and receives up to 10 passages (about 8,000 bytes) with exact citations. Before a citation is relied on, it is checked to be still current.

Use 2: requirement assessment combines a graph with RAG

When an analyst asks which systems a requirement changes, requirement-portal calls /internal/architecture/assess. Several "lanes" each contribute evidence, and an LLM weighs them:

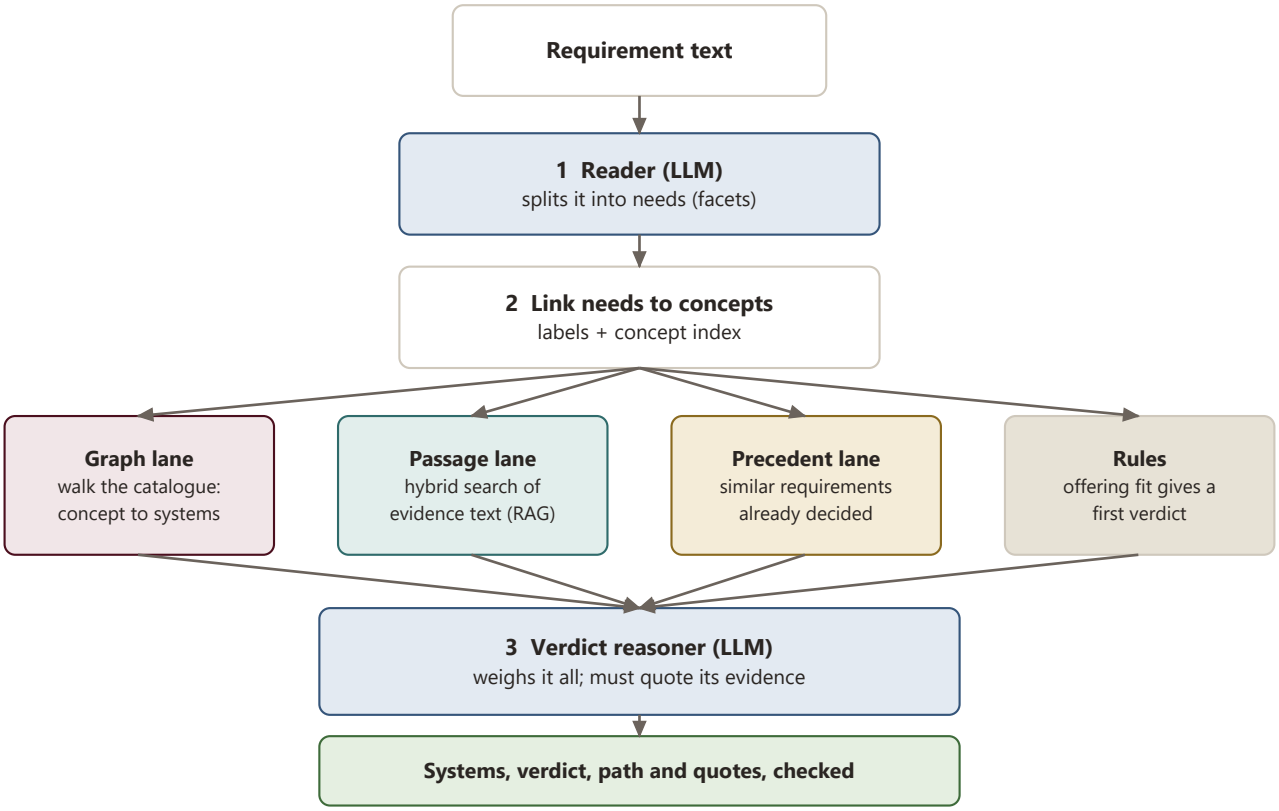


Figure 10. The assessment: an LLM reads the requirement, curated links reach concepts, four lanes gather evidence, and a second LLM decides, quoting what it used.

- **Graph lane**: walks the curated catalogue from concepts to the systems that realise them, and on to the owners and readers of any data or interface that changes.
- **Passage lane**: RAG over the catalogue's evidence documents, reranked (BM25 by default, a cross-encoder is optional) within a budget per need, so one wordy need cannot crowd out the rest.
- **Precedent lane**: the most similar requirements that reviewers already decided, shown to the reasoner as worked examples.
- **Rules**: how well the requirement fits an existing offering gives a first verdict.

The verdict reasoner must quote its evidence. An answer that names a system outside the release, or quotes text that is not in its evidence, is rejected.

Use 3: golden sets gate every retrieval change

ADR-0114 makes it a rule: every retrieval change is scored on a golden set before it ships, with fake models in the test suite and with real models by hand.

Golden set	What it tests	Results with the fake models
Impact set: 59 requirements labelled with verdict, offering, systems and concepts	The requirement assessment of Use 2	Today's simple matcher found 8% of expected systems. The assessment with the graph lane finds 33%, and verdict accuracy rose from 47% to 71%.
Library set: 14 documents in 7 formats across 7 products, 20 questions	The library search of Use 1	Recall@10 90%, MRR 84 to 90%, zero hits 10%. It found that search keeps 3 chunks per document, so a Word guide's three paragraphs push out its own table row that held the answer.

> **Watch out**
>
> All of these numbers come from the **fake** offline models, and the library questions are fixtures written by the team, not real searches. They are floors that stop regressions. **HYPOTHESIS**: real embeddings will score higher. That has not been measured yet.

Why we chose each piece

Choice	Why
RAG instead of retraining a model	Our documents change; approvals must take effect at once; answers must cite sources; and we can switch LLM providers.
A human reviews every block before publishing	Our rule: nothing AI-proposed or machine-extracted is published without a person accepting it. A bad extraction caught at review never reaches an answer.
Structure-aware chunking	Business documents are tables, slides and sections. Cutting along their structure keeps rows and paragraphs whole and gives exact citations.
Hybrid search with RRF	Our questions mix meaning ("scrambled") with exact codes ("XGPON", "SIP Trunk 30"). Neither search alone handles both.
Vectors in PostgreSQL (pgvector)	One database for records, chunks and vectors: fewer moving parts to run, back up and keep consistent.
A curated graph beside vectors	Requirements describe needs, not systems; impact follows relationships; reviewers need an explainable path. Measured: name and embedding matching alone found about 8% of expected systems.
Curated links, not an LLM-built graph	A wrong link would change which systems are flagged. Links must be accepted by a maintainer (ADR-0114).
Models behind ports, with a fake	Providers can change by configuration; every test runs offline and free.
Golden sets with floors in CI	Every tuning step is measured, and a change that makes retrieval worse fails the build.

What comes next

1. Run both golden sets with the real embedding and chat models and record the first live baseline.
2. Fix the finding above: let a document's best table row compete with its prose, for example with a reranker or a smarter per-document limit, and prove it on the library golden set.
3. Grow the library golden set with questions knowledge admins really ask, marked as real evidence.

11. Glossary

Term	Meaning
Attention	The mechanism inside an LLM that lets each word it writes weigh every word of the prompt.
Block	A piece of extracted text with its location: a paragraph, table row, slide or page.
Chunk	A small piece of an approved block, the unit that is embedded, searched and cited.
Citation	A pointer from an answer to the exact passage it used.
Context window	The most text an LLM can read in one prompt.
Corpus	The full set of approved documents a RAG system may search.
Cosine similarity	A closeness score between two vectors, from their angle: 1 is the same direction.
Cross-encoder	A reranking model that reads the question and a passage together to score them.
Embedding	A list of numbers (a vector) that places a text on a map of meaning.
Fine-tuning	Further training of a model to change its behaviour or style.
Golden set	Test questions with expert-approved answers, used to measure a system.
Graph	Nodes joined by named edges. A knowledge graph holds business facts and relationships.
Hallucination	A fluent, confident answer that is not supported by any source.
HNSW	Hierarchical Navigable Small World: a structure for finding nearest vectors fast.
Hybrid search	Vector search and keyword search combined.
Index	The stored chunks, vectors and keyword structures that make search fast.
LLM	Large language model: a model that writes text by predicting the next word.
MRR	Mean reciprocal rank: the average of 1 over the rank of the first right result.
pgvector	A PostgreSQL extension that stores vectors and searches them.
Prompt	The text given to an LLM: instructions, passages, question.
RAG	Retrieval-Augmented Generation: search first, then let the LLM answer from what was found.
Recall@k	The share of expected results that appear in the top k.
Reranker	A second, more careful scorer that reorders the top search results.
RRF	Reciprocal Rank Fusion: merges ranked lists by adding 1/(60 + rank).
Token	The unit LLMs read and are priced in: a word or part of a word.
Top-k	The k best-scoring results of a search.
Vector	An ordered list of numbers; here, an embedding.